AUTOMATED FAULT DETECTION METHOD FOR ROTATING MACHINES

[Motor Vibration Signal Process via MATLAB]

by

team_Predefined

Project

Course No: MATH 4421

Random Signal Processing

Student ID & Name

160021016 Fatema Zerin Prottasha

160021018 Mohammad Mahmudul Hasan

A Brief History on Machine Fault Type & Analysis

Machine fault was diagnosed earlier by the expert systems. A method known as **OFF-line fault detection** was carried out by experienced experts who had the knowledge and could guess that a particular fault is going to occur by manually analyzing the machine's overall condition.

Undoubtedly, this method could never render an authentic result, since based on human obtuse analysis, moreover, experts in different kinds of machine maintenance were widely unavailable for the time being. That is why we introduced various analytic methods with real time monitoring in accordance with :

- *Vibration monitoring*
- *Acoustic emission monitoring*
- *Air-gap eccentricity monitoring*
- *Partial discharge monitoring*
- *Magnetic Flux Monitoring*
- *Current monitoring*
- *Voltage monitoring*

As we are working on mechanical faults with vibration data, *Current, Voltage, Magnetic Flux & Partial Discharge Monitoring* are not taken into consideration because they denote ***Electrical Faults***. Besides, these monitoring methods and required equipments are *higly expensive*.

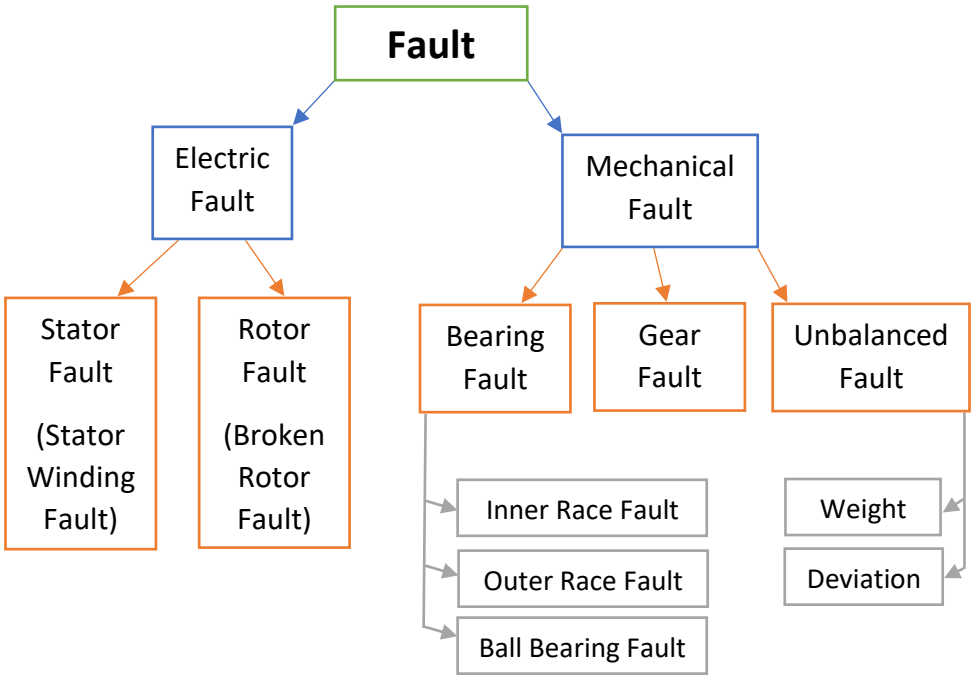
Above all, **Vibration analysis** is a wise choice for monitoring machines because this process is **generalized** for a number of fault types and more importantly, **economically efficient**. In addition, this gives us reliable data which can be depended on for precise detection of the particular fault type.

Since all the rotating machines cause huge amount of **noise and vibration** to a conspicuous amplitude, researchers on this field could differentiate several types of faults taking only **vibration parameters.** This vibration originates from magnetic, mechanical and aerodynamic reasons.

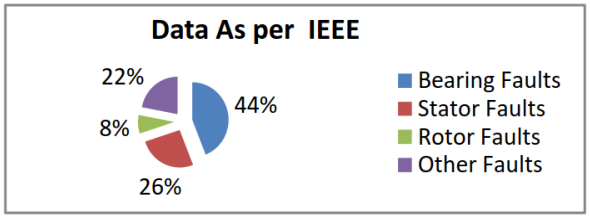
Now, to talk about **Vibration analysis,** first comes the sensors used to sense the machine vibration i.e. **accelerometers** which are installed onto the fault prone locations of the on-test machine, and when the machine is activated, accelerometer generates typical located vibration data over time domain. Acquired data from sanctioned accelerometers are like :

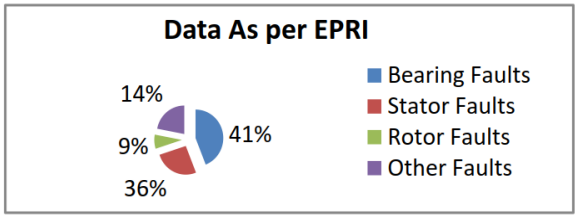
Analysis Technique	*Units*	*Data Typically Used*	*Use*
Acceleration versus Time	g or fraction of g	OARE SAMS	precise accounting of variation of acceleration magnitude as a function of time
Interval Average Acceleration versus Time	g or fraction of g	SAMS	indication of net accelerations lasting for time period ≥ interval parameter
Interval RMS Acceleration versus Time	g or fraction of g	SAMS	measure of oscillatory content in data
Trimmean Acceleration versus Time	g or fraction of g	OARE	smooth data and reject transient, higher magnitude contributions
Quasi-steady Three-dimensional Histogram	percentage of time and g or fraction of g	OARE	summarize acceleration vector magnitude and direction
Mapping of OARE Data	g or fraction of g	OARE	predict quasi-steady environment at locations other than OARE sensor
Power Spectral Density versus Frequency	g^2/Hz	SAMS	estimate of distribution of energy with respect to frequency
Cumulative RMS Acceleration versus Frequency	g_{RMS}	SAMS	quantifies contributions of spectral components to overall RMS acceleration level for time period studied in cumulative fashion
RMS Acceleration versus Frequency	g_{RMS}	SAMS	quantifies contributions of spectral components per frequency bin
Spectrogram	g^2/Hz	SAMS	road map of how acceleration signals vary with respect to both time and frequency

[1]



The figure below is the percentage failure components of Induction Motors :





[2-4]

From the diagrams above, it is evident that most of the faults occur due to mechanical drawback.

Type of vibration data analysis :

1) RMS of vibration amplitude
2) Time-Domain Analysis
3) Frequency Analysis (FFT)
4) Time-Frequency Analysis Method
5) Higher Order Statistics

Fault detecting method :

1. Model Based Approaches
2. Trending
3. Fault Thresholds
4. Multi-Dimensional Space Techniques

According to the parameters and method, sampled data from factory original should be taken as healthy vibration data so that those sets of vibration variables can be compared later to detect whether any fault has already occurred, or if it is in risk.

Variables & Parameters for Different Fault Types

Here goes the parameters and variables which will get into analysis of the fault type detection. All these parameters are based on machine vibration properties and output. To note, we are taking into consideration only **rotating machines** i.e. **motors**.

Type Of Fault	Sub Type	Peak Phase Match	Order FFT	Peak number Difference	Relative fault RMS
Bearing fault	Inner Race	0	2.4 ~ 18.5	feasible	>2 times
	Outer Race	0	3.5 ~ 21	feasible	>3.5 times
	Ball Bearing	0	2 ~ 19	Not feasible	~2 times
Mechanical Fault	Unbalance	1~2	0.5 ~ 1	Almost same	1 (appx)
	Horizontal Misalignment	0 ~ 3	1 ~ 2.6	Almost same	1 (appx)
	Vertical Misalignment	0 ~ 3	0.5 ~ 2.3	Slightly feasible	1 (appx)
Gear Fault		0 or 1~2	<=0.7	0	Lower

Fault Detection Algorithm

- Taking **input of Sample healthy** machine accelerometer data
- Taking input of to-be-tested or Faulty machine accelerometer data
- **Time domain** analysis (plotting Vibration Amplitude w r t time)
- Calculating **RMS values** of both the normal and fault vibration
- Fast Fourier Transform for **frequency domain** analysis (Plotting gain of vibration amplitude w r t frequency)
- RPM **order calculation** (measuring for which modulated order of frequency the fault vibration shows fault symptoms)
- Counting the **number of peaks** of both the normal and faulty FFT graph and comparing them
- Matching **the phase of normal and faulty peak values** (by peak coordinate indexing)
- Creating **conditions** for different faults according to the variables and parametric table data (experimental statistics and test drives)
- **Correlating** both the data and condition **to declare the exact type of fault** from the given set of faults.

N.B. This algorithm works on a specific type of machine for a fixed RPM and particular loading condition.

Solution: to overcome this consistency, further multi-axial accelerometer data should be processed and combined. Furthermore, drawbacks of FFT can be overcome by Short Term Fourier Transform (STFT) or, in failure of that, Wavelet Transform (WT).

Function & core Codes : MATLAB

```
clear()
clear all
close all

%% Asking to user for accelerometer record data already stored as .mat
file
disp('You are going to choose the mat file for Healthy Machine Data ')
pause(3)
uiopen
temp1=who; %display workspace variables
disp(temp1)
A0=input('Enter Healthy vibration data: ');

disp('You are going to choose the mat file for Fault verification Data ')
pause(3)
uiopen
temp2=who; %display workspace variables
disp(temp2)
F0=input('Enter faulty vibration data: ');



%% taking the accelerometer data record duration
Ti=input('DO you know the time duration? (Y/N) ', 's');

if Ti=='Y' || Ti=='y'
    T= input('Accelerometer data record duration (sec) :  ');
    Fs=input('Sampling Frequency (Hz) [Enter either the variable name or
the value you know] : ');
    t0=0:1/Fs:(T*Fs-1)/Fs;                    % normal data time range
    t1=0:1/Fs:(T*Fs-1)/Fs;                    % Fault data time range
    A0=A0(1:(Fs*T));                          % Normal data Vib Amp
    F0=F0(1:(Fs*T));
    L=Fs*T;
elseif Ti=='N' || Ti=='n'
    L1=length(F0);                             % Length of the Vib
Matrices
    L2=length(A0);

    if L1>L2
        L=L2;
    else
        L=L1;
    end
    Fs=input('Sampling Frequency (Hz) [Enter either the variable name or
the value you know] : ');   % freq of the acc record
    T= L/Fs;                      % time calculated from record data
length over sampling freq
```

```
% calculation of input argument
    t0=0:1/Fs:(T*Fs-1)/Fs;                          % normal data time range
    t1=0:1/Fs:(T*Fs-1)/Fs;                          % Fault data time range
    A0=A0(1:(Fs*T));                                % Normal data Vib Amp
    F0=F0(1:(Fs*T));                                % Fault data Vib Amp
end


[M,f0,P0,P1,val0,val1,fidx0,fidx1,cn0,cn1]=bellDetect(A0,F0,L,Fs,t0,t1);

%% RMS calculation

%Healthy
 normalRMS= rms(A0);

 bigFault=0;
% %InnerRace Fault
 IR_RMS= rms(F0);

 RMSratio=  IR_RMS/normalRMS;

 if normalRMS<IR_RMS
     bigFault= 1;
 end


%% phase measurement

countSim= 0;                    %initialising the var to count similar peak-
frequency
countSimExact=0;                %in case both the frequency and index matches
for i=1:cn0
    for j=1:cn1
        if round(fidx0(1,i))==round(fidx1(1,j))
            countSim= countSim+1;
        end
        if i==j && round(fidx0(1,i))==round(fidx1(1,j))
            countSimExact= countSimExact+1;
        end
    end
end



%% displaying the phase match result

fn= msgbox (sprintf('\n Normal VibrationRMS = %.3f \n Fault vibrationRMS =
%0.3f \n RMS ratio : %.3f \n\n Peaks exist at order %.1fx RPM \n Peak
number \n Healthy: %d  and faulty: %d \n \n Among %d of NormalPeaks and %d
of Fault peaks : \n %d peakFrequency(s) match and \n %d peak(s) exactly
match serially.',normalRMS, IR_RMS,
RMSratio,M,cn0,cn1,cn0,cn1,countSim,countSimExact),'Peak and RMS
information','help');
```

```
%% determinig fault type
bearing= false;
innerRace= false;
outerRace= false;
ballBearing= false;

gearFault= false;

mechaFault= false;
imbalance= false;
hMis= false;
vMis= false;


%% Fault verification

%counting FAult number
c=0;
%##########################################################################

% bearing fault detection
if bigFault==1 && countSim==0 && countSimExact==0 && M>2 &&
round(RMSratio)>=2
    bearing= true;
end

% ball bearing fault detection

if bearing==true && round(RMSratio)==2
     ballBearing= true;
     c=c+1;
end

% inner race fault detection
if bearing== true && ( ((2<=M && M<=3)&& round(RMSratio)>=4) || ((10<=M &&
M<=18) && round(RMSratio)>=4) )
    innerRace= true;
    c=c+1;
end

%outerRace fault detection
                                %loading condition
                                %deviation X
if bearing== true && (( (3<=M && M<=4) && (3<=RMSratio && RMSratio<=5.5) )
|| ( (M>=18.8) && (3<=RMSratio && RMSratio<=5.5) ))
    outerRace=true;
    c=c+1;
end

##########################################################################
%mechanical fault
%imbalance and misalignment (mecha) detection
if (0.1<=M && M<=2.5) && (0.98<=RMSratio && RMSratio<=1.1)
    mechaFault= true;
    c=c+1;
end
```

```
##########################################################################

% gear Fault detection
 if (M<0.7 && RMSratio<0.7) || (cn0==0 && cn1==0)
     gearFault=true;
     c=c+1;
 end


%% displaying Fault type

% msgbox editing struct variables
CreateStruct.Interpreter = 'tex';
CreateStruct.WindowStyle = 'modal';

x=0;
msg=cell(c,1);

if innerRace== true
         x=x+1;
         msg{x}=('\fontsize{20} \color{red} The given data shows INNER
RACE BEARING Fault ');
end

if  outerRace== true
         x=x+1;
         msg{x}=('\fontsize{20} \color{red} The given data shows OUTER
RACE BEARING Fault ');
end

if ballBearing== true
         x=x+1;
         msg{x}=('\fontsize{20} \color{red} The given data shows BALL
BEARING Fault ');
end


if gearFault== true
    x=x+1;
    msg{x}=('\fontsize{20} \color{red} The given data shows GEAR Fault ');
end

if mechaFault== true
    x=x+1;
    msg{x}=('\fontsize{20} \color{red} The given data shows MECHANICAL
Fault. Check the other sensor data for unbalace and misalignment.  ');
end

if bearing==false && gearFault==false && mechaFault==false
    x=x+1;
    msg{x}=('\fontsize{20} \color{green} The given data shows No known
Fault detected');
end
 msgbox(msg,'FaultType','error',CreateStruct)
```

~~~~ END ~~~~

[Function bellDetect]

```
function[M,f0,P0,P1,val0,val1,fidx0,fidx1,cn0,cn1]=
bellDetect(A0,F0,L,Fs,t0,t1)

%ploting time domain spectrum
figure
subplot(3,2,1)
plot(t0,A0,'g')
title('Healthy vibration vs Time')
xlabel('t(sec)')
ylabel('Vib Amplitude x g')

subplot(3,2,3)
plot(t1,F0,'r')
title('Fault vibration vs Time')
xlabel('t(sec)')
ylabel('Vib Amplitude x g')

subplot(3,2,5)
plot(t0,F0,'r',t1,A0,'g')
legend('Faulty','Healthy')
title('Time domain Spectra')
xlabel('t(sec)')
ylabel('Vib Amplitude x g')

hold on

%% Fourier Transform & Frequency spectrum
%Healthy
[f0,P0]=bellFFT(A0,L,Fs);
[f1,P1]=bellFFT(F0,L,Fs);
%Rms for frequency spectra
normalfRMS=rms(P0);
IR_fRMS= rms(P1);

subplot(3,2,2)
plot(f0,P0,'g')
title('Healthy vibration vs Frequency')
xlabel('frequency (Hz)')
ylabel('Gain(dB)')

subplot(3,2,4)
plot(f1,P1,'r')
title('Fault vibration vs Frequency')
xlabel('frequency (Hz)')
ylabel('Gain(dB)')

subplot(3,2,6)
plot(f1,P1,'r',f0,P0,'g')
legend('Fault','Healthy')
title('Frequency Spectra')
xlabel('frequency (Hz)')
ylabel('Gain(dB)')
```

```
%Peak Order Calculation
%Healthy
[Pord0]= bellPeak(P0,f0,2);
%Faulty
[Pord1]= bellPeak(P1,f1,2);

% fprintf("Number of peaks for healthy condition: %f \n",Pnum0)
% fprintf("Number of peaks for Inner Race Faulty condition: %f \n",Pnum1)
fprintf("Peak Orders(Freq) for healthy condition: %.3f \n",Pord0)
fprintf("Peak Orders(Freq) for faulty condition: %.3f \n",Pord1)

%Peak order comparison
M=Pord1./Pord0;

M=mean(M);
fprintf(" \n \n Peaks exist at order %.1fx RPM \n \n", M)

% Peak number determining and indexing
%%Healthy
fprintf("Healthy: \n \n")


PeakT0=max(P0)/pi;
pkFind=dsp.PeakFinder('PeakType','Maxima','PeakIndicesOutputPort',true,'Pe
akValuesOutputPort',true,'IgnoreSmallPeaks',true,'PeakThreshold',PeakT0);
[cn0,idx0,val0] = pkFind(P0);

[val0,idx0,fidx0]=nthMaxima(P0,cn0,f0)


%faulty
% cn0 is the number of healthy vib peaks

fprintf("Faulty: \n \n ")
PeakT1=PeakT0;
pkFind=dsp.PeakFinder('PeakType','Maxima','PeakIndicesOutputPort',true,'Pe
akValuesOutputPort',true,'IgnoreSmallPeaks',true,'PeakThreshold',PeakT1);
[cn1,idx1,val1] = pkFind(P1);

[val1,idx1,fidx1]=nthMaxima(P1,cn1,f1)

end
```

[Function bellFFT]

```
function [f,P1]=bellFFT(A0,L,Fs)

Y=fft(A0);                  %Compute the Fourier transform of the signal.
P2 = (abs(Y/L))*10;         %Compute the two-sided spectrum P2.
P1 = P2(1:L/2+1);           %Then compute the single-sided spectrum
                  %P1 based on P2 and the even-valued signal length L.
P1(2:end-1) = 2*P1(2:end-1);
f = Fs*(0:(L/2))/L;
end
```

[Function bellPeak]

```
function [Peakorders] = bellPeak(P0,f0,n)
 [spec, specOrder]= orderspectrum(P0, f0);

[~,Peakorders]=findpeaks(spec,specOrder,'SortStr','descend','NPeak',n);
Peakorders=round(Peakorders,3);

End
```

[Function nthMaxima]

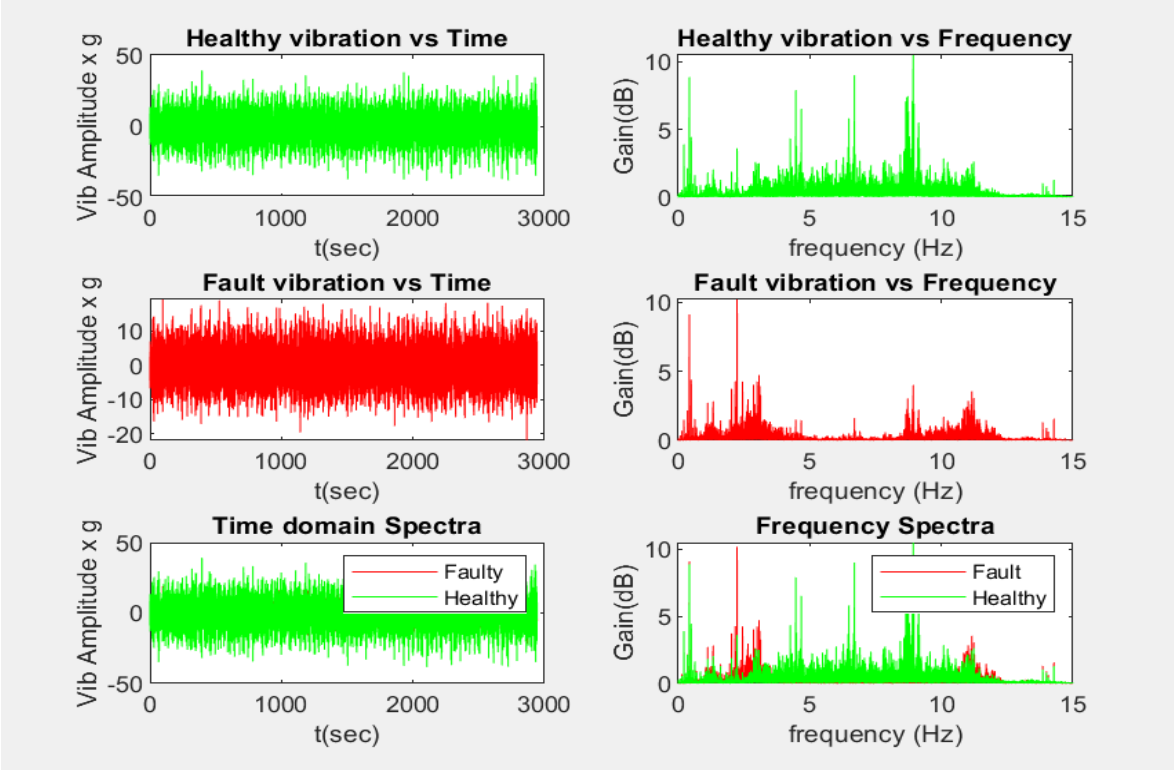
```
function[val,idx,fidx]=nthMaxima(P0,n,f0)              % arguments, (ins
values, number of peaks, frequency range)
%
%P0 is the array for searching maximus in there
%n is the number of top maxima to find
val=zeros(1,n); %defining array for reserving value of maximus
idx=zeros(1,n); %defining array for index of maximus
fidx=zeros(1,n);
for i=1:n
    [val(i),idx(i)]=max(P0);  %taking maxima value and indexing
    P0(idx(i))=-Inf;          %replacing maxima index with -Inf to search
for maxima again
end

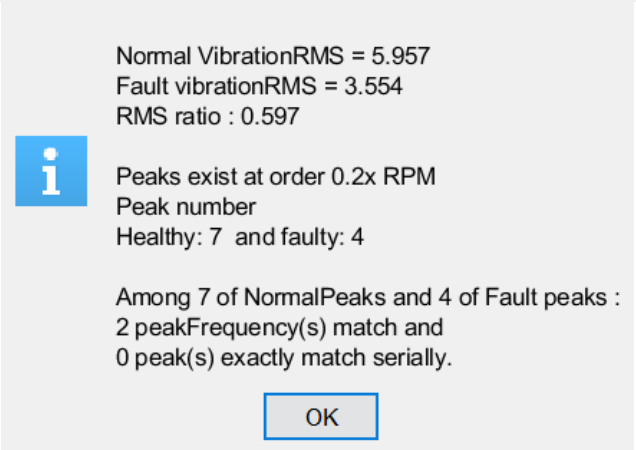
%recovering the replaced maxima value
for j=1:n
    P0(idx(j))=val(j);
    fidx(j)=f0(idx(j));
    fprintf (" \n The Amplitude value %.2f occurs at frequency %.3f Hz
\n", val(j),fidx(j))
end

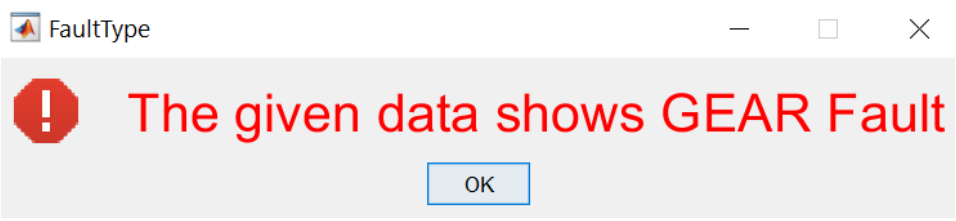
%done!!!!!!!!!!!!!!!!!!!!!!!!!!!!
end
```

Sample Figures of Processed Data and Information Text boxes

Here's what we get from running the MATLAB code to detect gear fault :







User Friendly Working-Interface

All that is done here is coder's work and not really comfortable for a random user to go in between the code lines and find what to run, where to give the input or declare which variable.

Keeping that in mind, we tried to make this coding and running environment as much user friendly as possible.

Unlike other codes where user has to know which variables the code needs to work in the inside, this code despite being a complicated one takes only :

Input Section :

- ✓ **Sample data** of normal machine vibration and fault vibration by **UI**, allowing the user to **directly access the directory** while running the code in background.
- ✓ Asking users of sampling **frequency**
- ✓ Giving the users choices to declare whether they know **time duration**. If yes, user can give any time that complies with *the vibration variable order/length*. If no, **the code itself calculates time** from length and sampling frequency.

Output Section :

- ✓ After having processed all the vibration data, it **directly declares the fault**
- ✓ All the necessary information those have detected the type of fault are shown altogether in **an information text box** (No need to scroll over the command window or workspace for the variable values)
- ✓ Although automated fault detection, the code also generates all the comparative and superposed graphs, so that the user can also comprehend the condition of the machine manually (**graphical representation**).

The whole methodology of fault detection can in all way be claimed UNIQUE !

Although vibration signal process in time and frequency domain, power spectral density and other extraction formats has been used by experts, faults are still detected manually. Through this code, we introduce a technique that after all the vibration analysis, the computer by means of MATLAB can **declare the type of fault on its own**.

This will give ease to the people and experts in maintenance duty, and another step forward into **automated predication system** in the field of machine health.

Manual fault detection along with electromagnetic sensing device results cannot always predict machine fault and failure in an authentic and impeccable manner. But vibration and noise*, inevitable signature of any machine* is an indomitable parameter to say not only the machine working phase in real time but also machine health condition.

Apart from laboratories and industrial purpose, this analysis with further precision to be made on *experimental statistics*, can help monitoring bulks of machines altogether, **locate faults of an humanly inaccessible location** and give prior warning of massive machine failure which can save money and life.

References :

[1] NASA Technical Report Server. Website : https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/19970034695.pdf

[2] IEEE, Electrical Power Institution, IEEE standard : 493-1997

[3] "IEEE Recommended Practice for The Design of Reliable Industrial and Commercial Power Systems", IEEE Std. 493-1997[IEEE Gold Book], Appendix H,1997.

[4] P.F. Allbrecht, J.C.Appiarius and R.M.McCoy, "Assessment of the Reliability of Motors in Utility Applications-Updated", IEEE transactions on Energy Conversion, Vol. 1,No. 1,pp. 39-46,1986.